

CST-407 Activity 3 Password Hashing and Cracking

Alex M. Frear

College of Science, Engineering, and Technology, Grand Canyon University

Course Number: CST-407

Professor: Dr. Melody White

08/04/2025

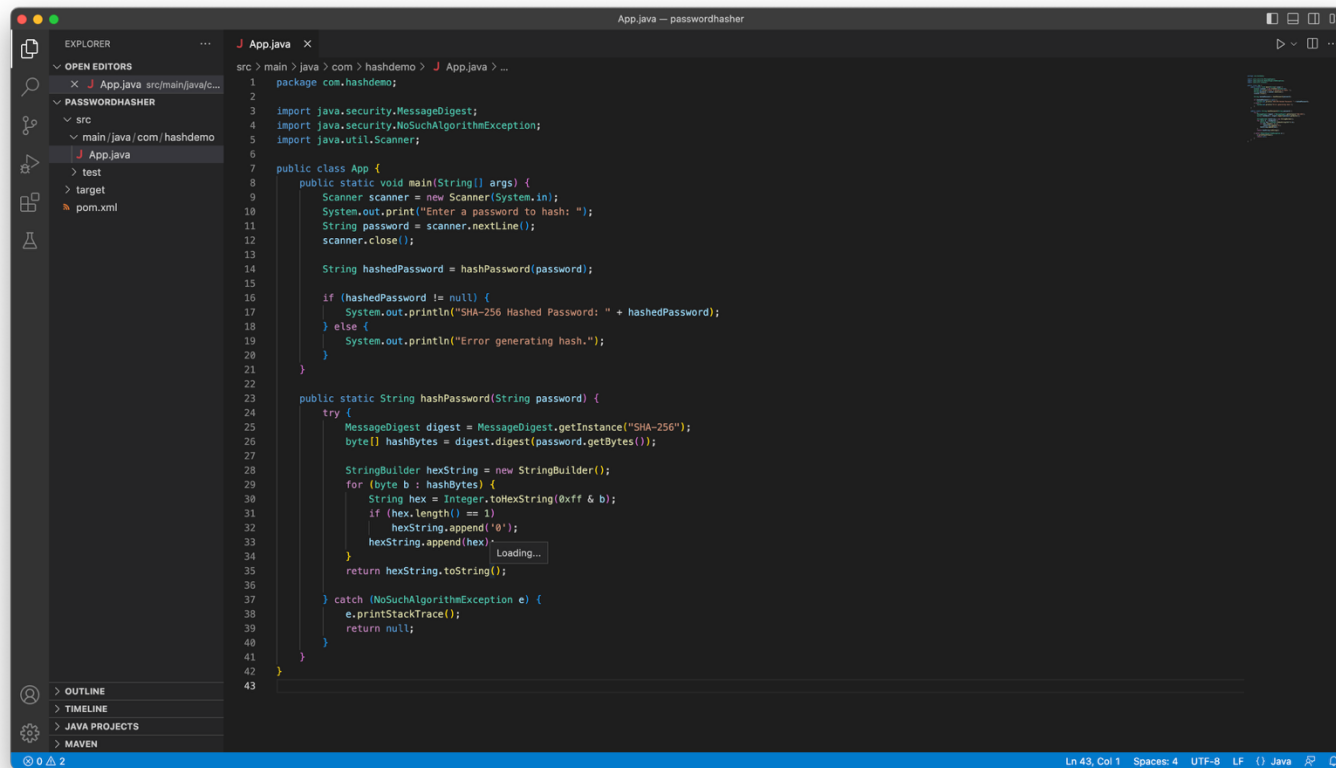
Table of Contents

Part 2: Write a Java Program That Hashes a Password	3
Screenshots	3
App.java Source Code	3
App.java Hashed Password Output.....	4
Part 3: Cracking the Hashed Password	5
Screenshots	5
App.java Password Input Prompt.....	5
App Hashing Prompt and MD5 Output	6
Selecting Brute-force Alphabet	7
Successful Brute-Force Attack Result.....	8
Summary of Key Concepts.....	9
Password Cracking Time Estimates and Best Practices	10
Password Cracking Time Estimate Table	10
Best Practices for Creating Secure Passwords.....	11

Part 2: Write a Java Program That Hashes a Password

Screenshots

App.java Source Code



```
1 package com.hashdemo;
2
3 import java.security.MessageDigest;
4 import java.security.NoSuchAlgorithmException;
5 import java.util.Scanner;
6
7 public class App {
8     public static void main(String[] args) {
9         Scanner scanner = new Scanner(System.in);
10        System.out.print("Enter a password to hash: ");
11        String password = scanner.nextLine();
12        scanner.close();
13
14        String hashedPassword = hashPassword(password);
15
16        if (hashedPassword != null) {
17            System.out.println("SHA-256 Hashed Password: " + hashedPassword);
18        } else {
19            System.out.println("Error generating hash.");
20        }
21    }
22
23    public static String hashPassword(String password) {
24        try {
25            MessageDigest digest = MessageDigest.getInstance("SHA-256");
26            byte[] hashBytes = digest.digest(password.getBytes());
27
28            StringBuilder hexString = new StringBuilder();
29            for (byte b : hashBytes) {
30                String hex = Integer.toHexString(0xff & b);
31                if (hex.length() == 1) {
32                    hexString.append('0');
33                }
34                hexString.append(hex);
35            }
36            return hexString.toString();
37        } catch (NoSuchAlgorithmException e) {
38            e.printStackTrace();
39            return null;
40        }
41    }
42
43 }
```

Figure 1 App.java file showing the Java source code that uses SHA-256 to hash a user-provided password.

App.java Hashed Password Output

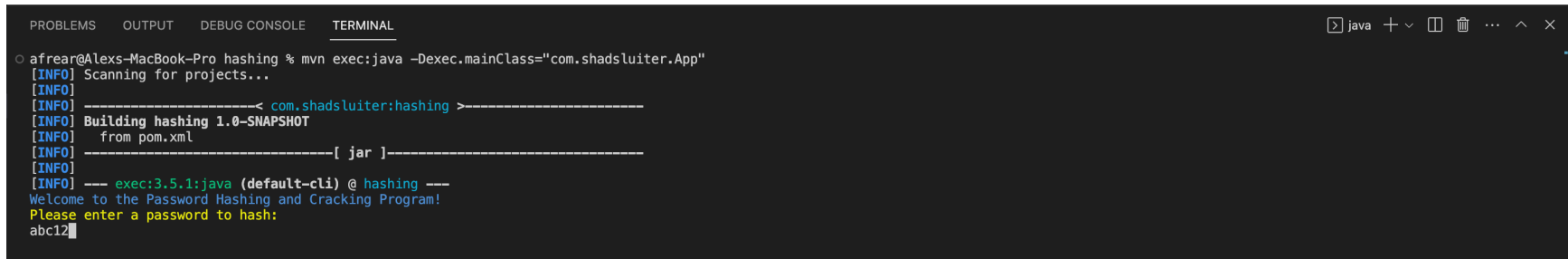
```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
afrear@Alexs-MacBook-Pro passwordhasher % mvn exec:java
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.hashdemo:passwordhasher >-----
[INFO] Building passwordhasher 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- exec:3.1.0:java (default-cli) @ passwordhasher ---
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-utils/3.4.2/plexus-utils-3.4.2.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-utils/3.4.2/plexus-utils-3.4.2.pom (8.2 kB at 26 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus/8/plexus-8.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus/8/plexus-8.pom (25 kB at 509 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-component-annotations/2.1.1/plexus-component-annotations-2.1.1.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-component-annotations/2.1.1/plexus-component-annotations-2.1.1.pom (770 B at 30 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-containers/2.1.1/plexus-containers-2.1.1.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-containers/2.1.1/plexus-containers-2.1.1.pom (6.0 kB at 208 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus/6.5/plexus-6.5.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus/6.5/plexus-6.5.pom (26 kB at 779 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-exec/1.3/commons-exec-1.3.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-exec/1.3/commons-exec-1.3.pom (11 kB at 366 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-parent/35/commons-parent-35.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-parent/35/commons-parent-35.pom (58 kB at 1.3 MB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-utils/3.4.2/plexus-utils-3.4.2.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-utils/3.4.2/plexus-utils-3.4.2.jar (267 kB at 4.2 MB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-component-annotations/2.1.1/plexus-component-annotations-2.1.1.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-component-annotations/2.1.1/plexus-component-annotations-2.1.1.jar (4.1 kB at 46 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-exec/1.3/commons-exec-1.3.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-exec/1.3/commons-exec-1.3.jar (54 kB at 1.6 MB/s)
Enter a password to hash: hunter2025
SHA-256 Hashed Password: ced7726c03423d4eb933a13260c4455fbb282ca775d4219bfa1ab4d1a47c756a
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 52.645 s
[INFO] Finished at: 2025-08-03T11:22:36-07:00
[INFO]
afrear@Alexs-MacBook-Pro passwordhasher %
```

Figure 2 The program was executed using Maven and successfully hashed the password hunter2025, producing a SHA-256 encoded output.

Part 3: Cracking the Hashed Password

Screenshots

App.java Password Input Prompt



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL
afrear@Alexs-MacBook-Pro hashing % mvn exec:java -Dexec.mainClass="com.shadsluiter.App"
[INFO] Scanning for projects...
[INFO] -----< com.shadsluiter:hashing >-----
[INFO] Building hashing 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO] --- exec:3.5.1:java (default-cli) @ hashing ---
Welcome to the Password Hashing and Cracking Program!
Please enter a password to hash:
abc12
```

Figure 3 The user entered a new password "abc12" to generate a hash using the Password Hashing and Cracking Program.

App Hashing Prompt and MD5 Output

```
Welcome to the Password Hashing and Cracking Program!  
Please enter a password to hash:  
abc12  
Which hash algorithm would you like to use? [MD5, SHA-1, SHA-256, SHA-512]. Leave blank for MD5:  
  
Hashed Password: shV+eyrnFqdHwXCx8e+3oA==  
Algorithm used: MD5  
-----  
Enter a hash to crack (Leave blank to use the generated hash):
```

Figure 4 User enters the password abc12 and selects MD5 as the hashing algorithm. The application returns the hashed value shV+eyrnFqdHwXCx8e+3oA== using MD5.

Selecting Brute-force Alphabet

```
Welcome to the Password Hashing and Cracking Program!
Please enter a password to hash:
abc12
Which hash algorithm would you like to use? [MD5, SHA-1, SHA-256, SHA-512]. Leave blank for MD5:

Hashed Password: shV+eyrnFqdHwXcX8e+3oA==
Algorithm used: MD5

Enter a hash to crack (Leave blank to use the generated hash):

Which algorithm should be used for cracking? [MD5, SHA-1, SHA-256, SHA-512]. Leave blank to use MD5:

Would you like to use a dictionary attack or a brute-force attack?
1: Dictionary Attack
2: Brute-force Attack
2
Brute-force Attack Selected
Select an alphabet for brute-force attack (Enter 0-14, Leave blank for 0):
0: abcdefghijklmnopqrstuvwxyz
1: ABCDEFGHIJKLMNOPQRSTUVWXYZ
2: 0123456789
3: !@#$%^&*()-_+=[]{}|;:,.<>?
4: abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ
5: abcdefghijklmnopqrstuvwxyz0123456789
6: abcdefghijklmnopqrstuvwxyz!@#$%^&*()-_+=[]{}|;:,.<>?
7: ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789
8: ABCDEFGHIJKLMNOPQRSTUVWXYZ!@#$%^&*()-_+=[]{}|;:,.<>?
9: 0123456789!@#$%^&*()-_+=[]{}|;:,.<>?
10: abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789
11: abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ!@#$%^&*()-_+=[]{}|;:,.<>?
12: abcdefghijklmnopqrstuvwxyz0123456789!@#$%^&*()-_+=[]{}|;:,.<>?
13: ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789!@#$%^&*()-_+=[]{}|;:,.<>?
14: abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789!@#$%^&*()-_+=[]{}|;:,.<>?
5
```

Figure 5 User selects a brute-force attack and chooses option 5 (lowercase letters + numbers) for the character set, which matches the format of the password abc12.

Successful Brute-Force Attack Result

```
Welcome to the Password Hashing and Cracking Program!
Please enter a password to hash:
abc12
Which hash algorithm would you like to use? [MD5, SHA-1, SHA-256, SHA-512]. Leave blank for MD5:

Hashed Password: shV+eyrnFqdHwXcX8e+3oA==
Algorithm used: MD5

Enter a hash to crack (Leave blank to use the generated hash):

Which algorithm should be used for cracking? [MD5, SHA-1, SHA-256, SHA-512]. Leave blank to use MD5:

Would you like to use a dictionary attack or a brute-force attack?
1: Dictionary Attack
2: Brute-force Attack
2
Brute-force Attack Selected
Select an alphabet for brute-force attack (Enter 0-14, Leave blank for 0):
0: abcdefghijklmnopqrstuvwxyz
1: ABCDEFGHIJKLMNOPQRSTUVWXYZ
2: 0123456789
3: !@#$%^&*()-_+=[]{}|;:,.<>?
4: abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ
5: abcdefghijklmnopqrstuvwxyz0123456789
6: abcdefghijklmnopqrstuvwxyz!@#$%^&*()-_+=[]{}|;:,.<>?
7: ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789
8: ABCDEFGHIJKLMNOPQRSTUVWXYZ!@#$%^&*()-_+=[]{}|;:,.<>?
9: 0123456789!@#$%^&*()-_+=[]{}|;:,.<>?
10: abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789
11: abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ!@#$%^&*()-_+=[]{}|;:,.<>?
12: abcdefghijklmnopqrstuvwxyz0123456789!@#$%^&*()-_+=[]{}|;:,.<>?
13: ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789!@#$%^&*()-_+=[]{}|;:,.<>?
14: abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789!@#$%^&*()-_+=[]{}|;:,.<>?
5
=====
Trying length: 1 which will have 36 combinations
=====
Trying length: 2 which will have 1,296 combinations
=====
Trying length: 3 which will have 46,656 combinations
=====
Trying length: 4 which will have 1,679,616 combinations
=====
Trying length: 5 which will have 60,466,176 combinations
Brute Force Attack Result: abc12 is the password. shV+eyrnFqdHwXcX8e+3oA== is the hash value. Which was hashed with MD5

Would you like to hash another password? (yes/no):
```

Figure 6 The program attempts combinations from length 1 through 5 using the selected character set (lowercase + numbers). The brute-force attack successfully cracks the MD5 hash and reveals the original password abc12. The hash value shV+eyrnFqdHwXcX8e+3oA== matches the one generated during hashing.

Summary of Key Concepts

This was my first time ever trying password hashing and cracking, and it was honestly a lot more eye-opening than I expected. I had heard of brute-force attacks before, mostly in general conversations about cybersecurity, but I didn't really know how they worked in practice. I also wasn't familiar with concepts like hashing algorithms (MD5, SHA-1, etc.) or how hashed passwords are actually stored and protected. Going through this activity gave me a hands-on look at how these systems work and why they're important.

Earlier in the activity, I tested a longer password (Hunter2025) just out of curiosity, and the cracking process ran for hours without success. In contrast, the final test I documented used a much shorter password (abc12), which was cracked in a matter of seconds. That difference really stuck with me. It showed me just how much a few extra characters or the use of upper/lowercase and symbols can change how secure a password is. It was a good lesson in why password complexity matters.

This experience helped me finally understand why cybersecurity experts always emphasize using strong passwords and why certain hashing algorithms like MD5 are no longer considered secure. Now that I've actually run a brute-force attack myself, I have a much better appreciation for both the risks and the protections involved in password security. This was a solid introduction to a topic I previously only knew by name.

Password Cracking Time Estimates and Best Practices

Password Cracking Time Estimate Table

Password Type	Example	Cracking Time Estimate	Notes
6 lowercase letters	kitten	Less than 1 second	Easily cracked with brute-force
8 lowercase letters	sunshine	~2 minutes	Slightly better but still weak
8 mixed-case letters	Summer19	~22 minutes	Adds complexity but still vulnerable
8 chars w/ letters & numbers	abc12345	~6 minutes	Common format; easily guessed
10 chars w/ mixed-case + num	P@ssw0rd21	~3 days	Better but predictable patterns can be cracked
12 chars w/ symbols	Tr!ckyP@ss2025	~34 years	Strong if not based on dictionary words
16 random chars (full set)	x7@B4#zP!9qT0\$Im	Trillions of years	Considered secure against brute-force

Best Practices for Creating Secure Passwords

This was my first time working with password hashing and cracking, and the experience was eye-opening. I originally used a stronger password like “Hunter2025,” which resisted brute-force attempts even after more than 16 billion guesses. Later, I tried a simpler password like “abc12,” and it was cracked almost instantly.

Seeing how quickly weak passwords fall to brute-force attacks helped me understand why strong passwords are so critical. Even small improvements , like adding symbols or increasing length , drastically improve a password’s strength.

Based on what I learned, here are a few best practices to follow:

- Use **12+ characters** with uppercase, lowercase, numbers, and special characters.
- Avoid common words, names, or obvious patterns like “123” or “password.”
- Use **passphrases** that are long but memorable (e.g., Ocean\$Blue!Horse47).
- Use a **password manager** to generate and store complex passwords securely.
- Never reuse the same password across multiple websites or apps.

These practices can make brute-force or dictionary-based attacks much less effective, helping keep your accounts and information safe.